

numba-cuda vs numba-cuda-mlir

Same 9 kernels, one source file, two compilers. Holding the GPU, CUDA 13.3 ptxas, workload and Python version constant — varying only the codegen frontend (NVVM vs MLIR). JIT time · PTX/SASS codegen · runtime & FPS · CUDA graphs.

bit-exact

OUTPUT EQUIVALENCE

$\approx 1024^2$

LAUNCH→COMPUTE CROSSOVER

parity

UNDER CUDA GRAPHS

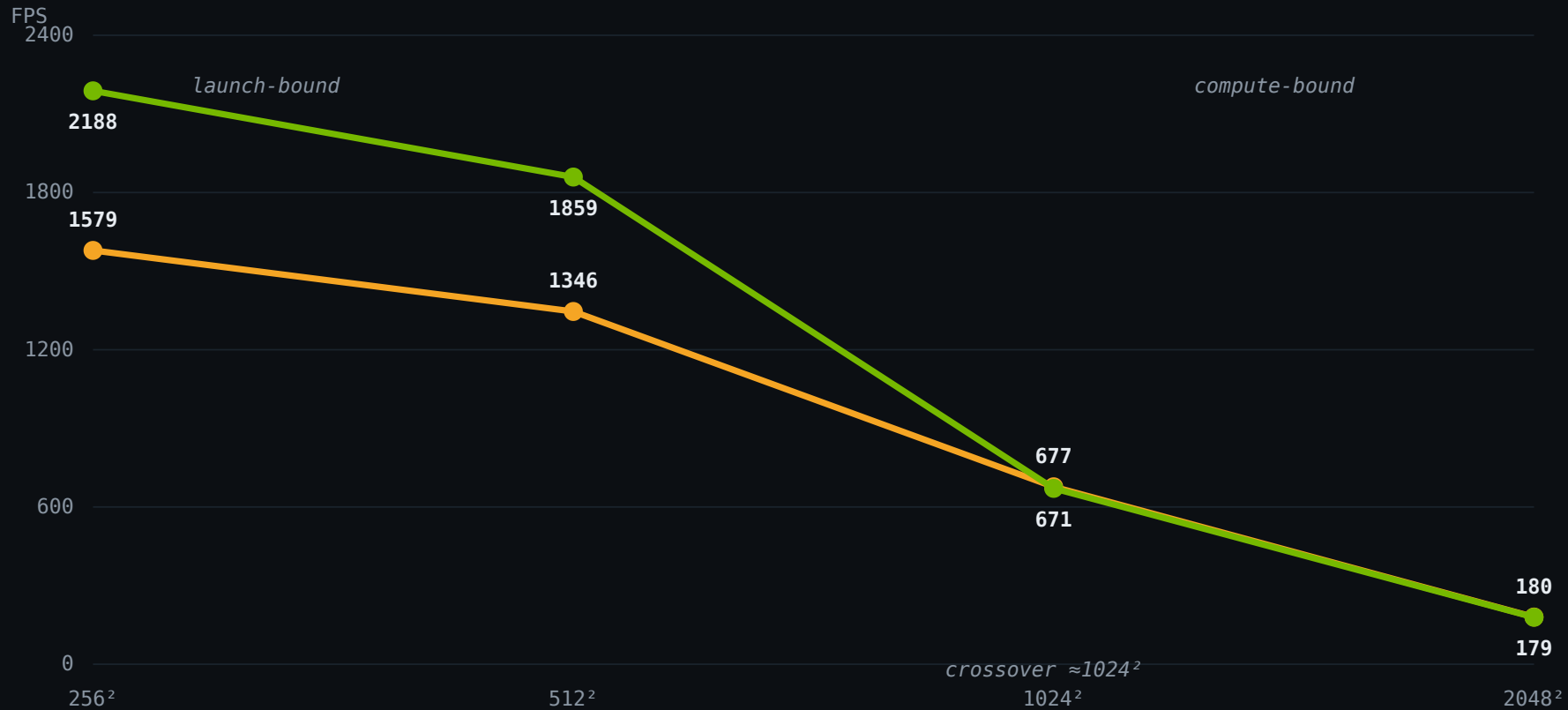
One variable: the compiler frontend

	CLASSIC	MLIR
package	numba-cuda 0.30.2	numba-cuda-mlir 0.4.0
frontend	Numba IR → LLVM → NVVM → PTX	MLIR → PTX (LTOIR)
opt / LTO default	NVVM opt-3, no LTO	LTO, opt>0
Python	3.12.13	3.12.13
cuda-bindings / ptxas	13.3.1 / CUDA 13.3	13.3.1 / CUDA 13.3
GPU	RTX 5880 Ada (sm_89), driver 595.71.05	

Bit-exact output on all 9 kernels at 256^2 / 512^2 / 1024^2 / 2048^2 → a fair race on identical work.

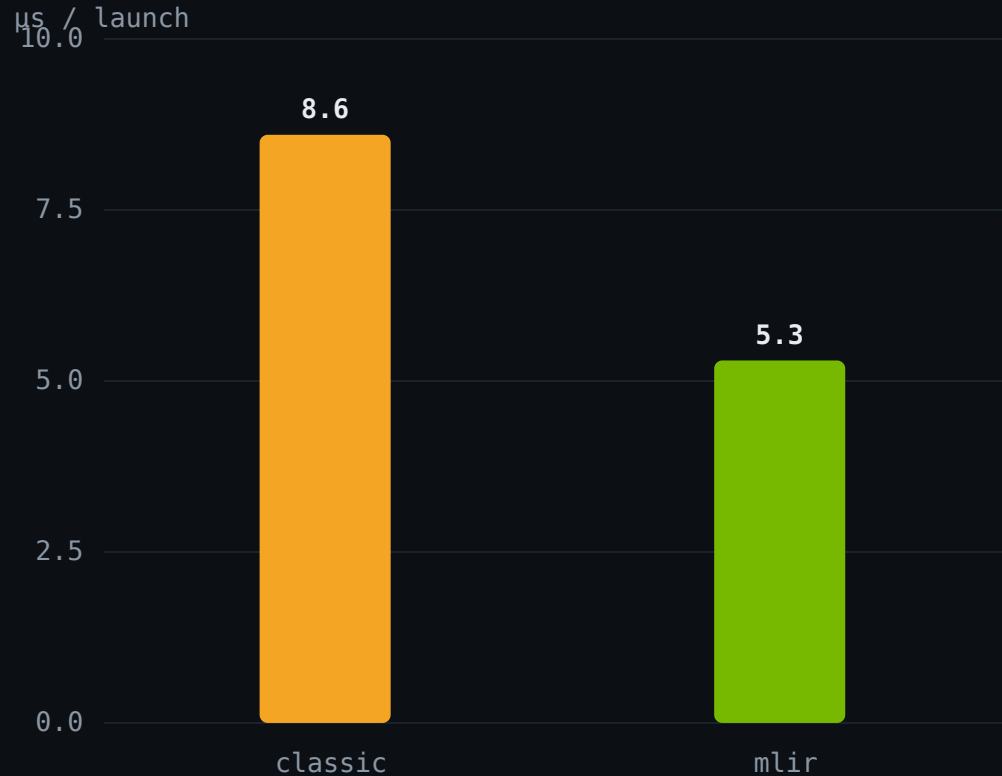
The launch-bound → compute-bound crossover

■ classic (NVVM) ■ mlir (MLIR)



Below $\sim 1024^2$ the sim is **host-launch-bound** (37 tiny launches/frame) and mlir's lighter dispatch wins $\sim 38\%$. At/above the crossover the GPU dominates and the two are **identical**.

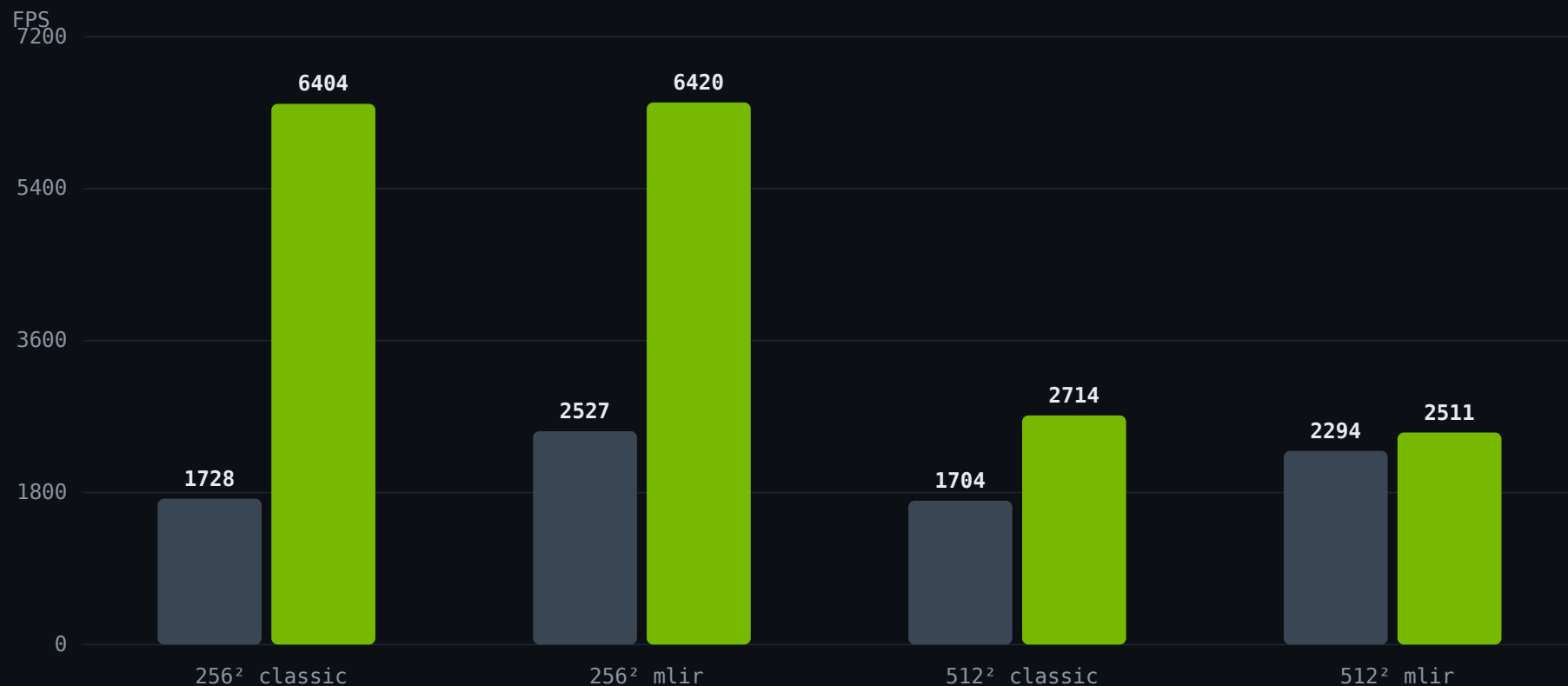
Equal GPU time; unequal dispatch cost



- ▶ Pure GPU time/frame is **equal** — nsys 0.34 ms both at 512²; ncu: 30/30 regs, ~77% occupancy, 79% SM both.
- ▶ The gap is entirely the **Python** → **driver launch path**: **8.6 μs** vs **5.3 μs** per launch.
- ▶ × 37 launches/frame ≈ the measured wall-vs-GPU gap. mlir dispatches ~38% cheaper.

CUDA graphs erase the gap

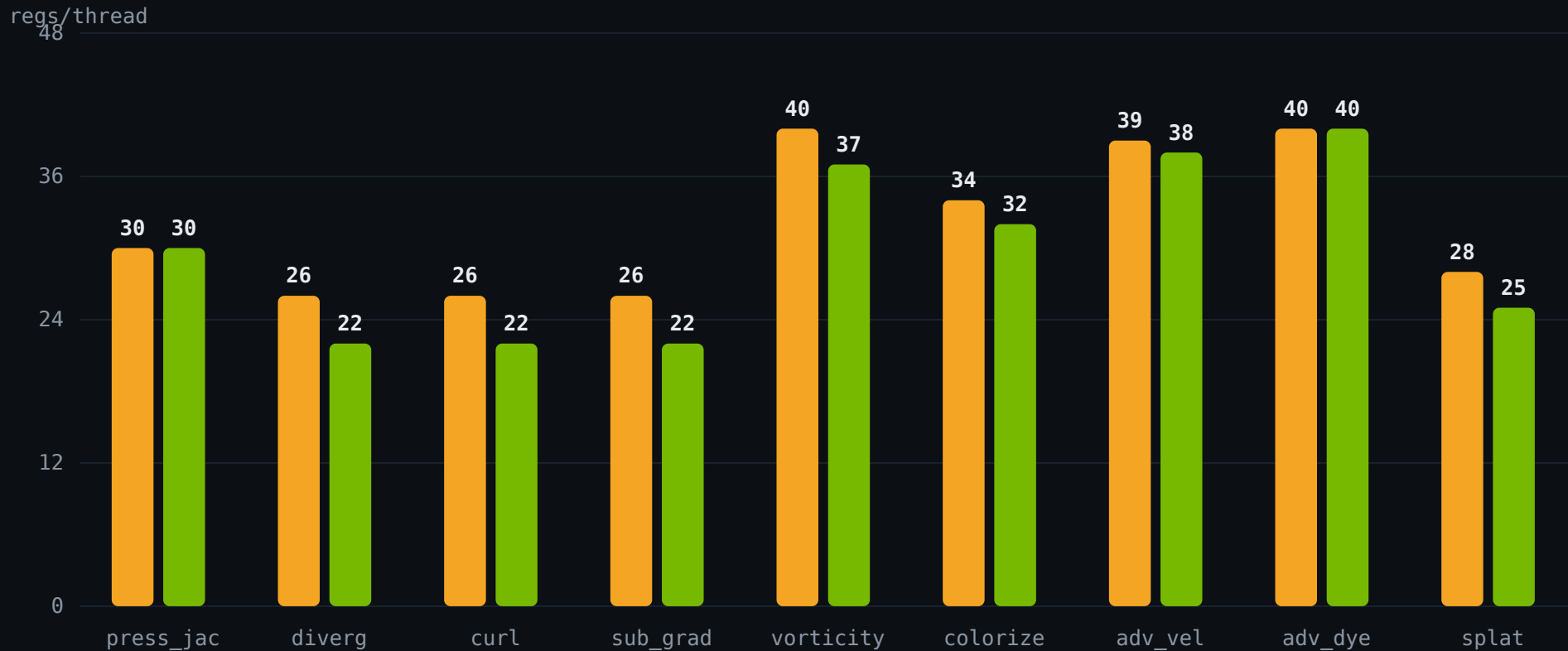
■ eager ■ graphed



Capture the 37 launches once, replay as one graph launch. Both backends **converge to the same FPS** (classic gains **3.7×** vs mlir 2.5× – it had more overhead to remove). mlir's edge is an eager-path effect only.

mlir slightly tighter by default — but see slide 08

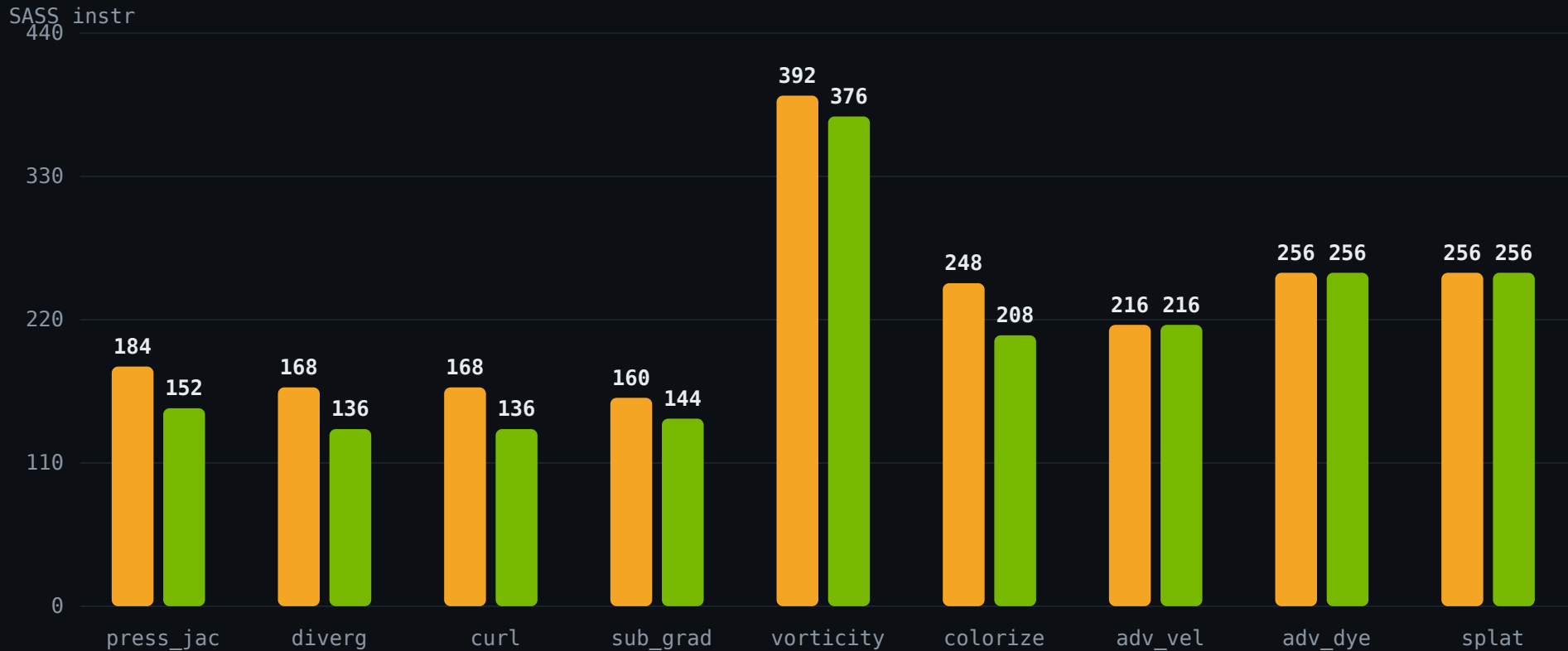
■ classic ■ mlir



All under the ~42-register occupancy threshold at 16×16 → these deltas do **not** change occupancy. Dominant **pressure_jacobi** is 30 = 30.

Fewer instructions $\neq$ faster here

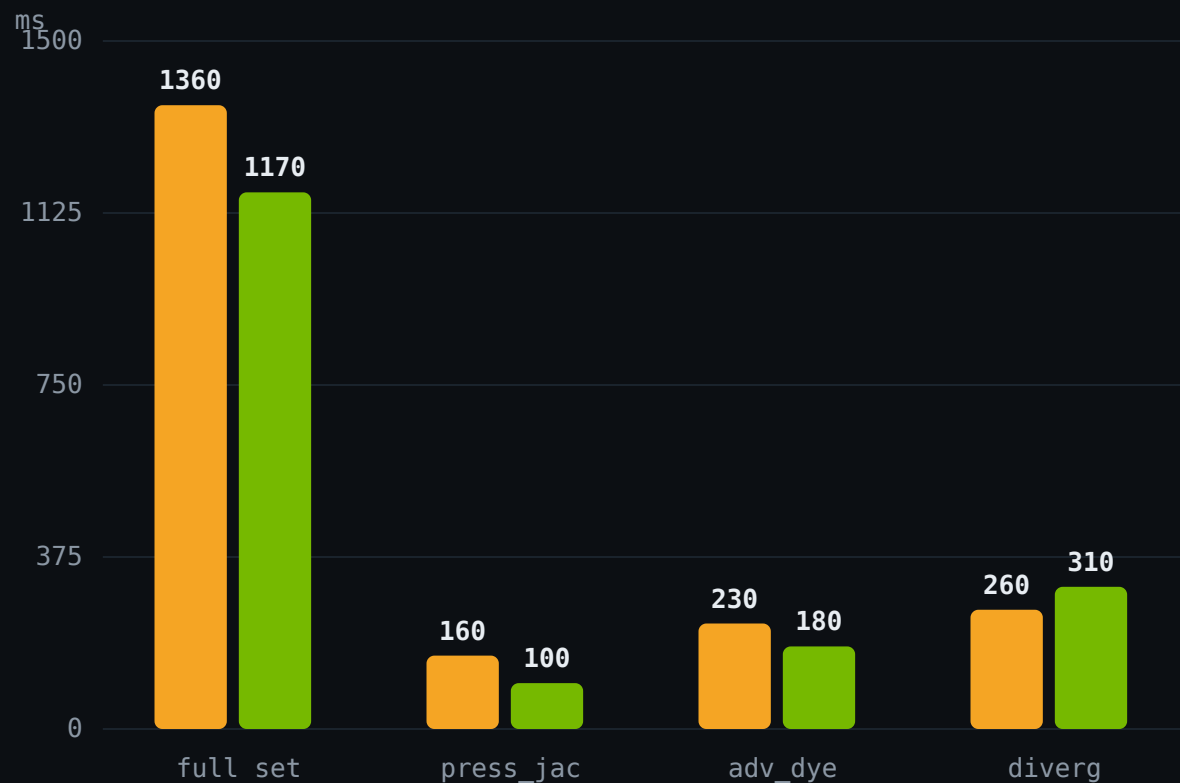
classic mlir



mlir is -10-19% on the stencil kernels, classic wins colorize – yet **GPU runtime is identical**: these kernels are **float64-throughput bound** (Python literals promote to f64), not instruction-issue bound.

mlir compiles ~14% faster

■ classic ■ mlir



1.36s

CLASSIC – FULL SET

1.17s

MLIR – FULL SET (3.12)

Context pre-warmed → pure compile. No on-disk cache, so paid every process.

The tightness is LTO, not the frontend

PRESSURE_JACOBI REGS	LTO OFF	LTO ON
classic (NVVM)	30	30
mlir (MLIR)	50	30

- ▶ At matched LTO the frontends are identical — 30 = 30.
- ▶ mlir's default-config edge is the **LTO pipeline**; the MLIR frontend alone is **worse** (50).
- ▶ classic's NVVM reaches 30 **without** LTO.

Same machine code; different launch path

VECTOR	WINNER	MAGNITUDE
equivalence	tie	bit-exact, all sizes
GPU codegen (runtime)	tie	equal GPU time / occupancy
static SASS (default)	mlir	-10-19% – but = at matched LT0
JIT compile time	mlir	~14% faster
eager FPS < 1024 ²	mlir	+38% (5.3 vs 8.6 μ s/launch)
FPS $\geq$ 1024 ² · with graphs	tie	compute-bound parity

Pick on launch model, not codegen

- ▶ The two compilers generate **equivalent GPU code** — bit-exact, equal GPU runtime, identical occupancy; the SASS edge is an LTO effect classic matches at equal LTO.
- ▶ The only real-world difference is **eager-mode launch overhead** — mlir ~38% cheaper, matters only below $\sim 1024^2$, and **vanishes under CUDA graphs**. mlir also compiles ~14% faster.
- ▶ Backend-independent win #1: kernels run in **float64** (Python literals) — **float32 literals** would cut arithmetic on both.
- ▶ Backend-independent win #2: launch-bound below $\sim 1024^2$ — **CUDA graphs** roughly **triple FPS** on both.